A Appendices

A.1 Reliability

EMDedup backend uses RDMA unreliable datagram (UD) for all all-to-all communications and enforces reliability.

REQ loss in input all-to-all. Although the switch never drops REQs, RDMA reliable connection (RC) is infeasible when in-switch deduplication is employed. RC transparently retransmits lost packets using headers identical to original ones, which introduces a fatal logical error. If the first occurrence of a key is lost during downlink transmission to the destination rank, the switch marks all subsequent duplicate occurrences of that key as `Invalid`. When the RC retransmitted REQ later arrives at the switch, it is still tagged as `Invalid` according to the key state. Consequently, the destination GPU discards the retransmitted packet, leading to permanent loss of the sparse feature and incorrect model computation. EMDedup eliminates the issue by maintaining a sliding window at the backend, which buffers all outbound REQs in strict sending order. Each sending rank maintains an individual bitmap for every sent REQ, where each bit index corresponds to a unique destination rank ID and is initially set to mark pending packet reception for that rank. Upon receiving an ACK whose echoed `Seq` matches a sent REQ, the sender clears the bit corresponding to ACK's source rank in the bitmap. After completing the current minibatch, the sender retransmits all REQs with uncleared bitmap bits following the original sending order. These retransmitted packets are tagged with a distinct `Class` value (*e.g.*, 3) to differentiate them from normal original transmissions. The switch will ignore them and forward retransmitted REQs directly, bypassing all deduplication logic.

RESP loss in output all-to-all. Recovering RESPs is similar to recovering REQs, except for RESPs can be multicast by the switch. For a multicast RESP, EMDedup must record all destination ranks for that packet and set the corresponding bits in that packet's bitmap. When receiving ACKs, a fully cleared bitmap indicates a multicast RESP has been successfully received by all destination ranks. Otherwise, the sending rank retransmits the packet to the destination ranks equal to those uncleared bit indices.

Gradient loss in gradient all-to-all. Because EMDedup works seamlessly with ATP for in-switch gradient aggregation, only updating the bitmap field in ATP packets with the state bitmap, we rely on ATP's loss recovery to guarantee gradient data reliability [31].

A.2 Conventional Three-round Input All-to-alls

To realize conventional three-round input all-to-alls, PyTorch uses the unified `All-to-All` interface with four parameters: `input_buffer`, `input_split_array`, `output_buffer`, and `output_split_array`, where `split_array` partitions `input_buffer` or `output_buffer` across all ranks, respectively, and explicitly specifies the exact data buffer range that each rank sends and receives from every peer. For example, consider a 4-rank deployment with batch size $B = 2$ samples per rank and four EMTs sharded with table-wise (one table per rank) parallelism. Rank 0's `Length` array is $[2, 3,\ 3, 4,\ 4, 2,\ 2, 3]$ (*i.e.*, table-major) and `Value` array contains 23 indices.

Round 1 Split all-to-all. Rank 0 computes the subarray size that the rank would send to each peer rank: $[\,5,\ 7,\ 6,\ 5\,]$ value counts (sum of lengths per table) and batch size B=2 for each rank, encoded as a 12-element vector $[2, 5, 2,\ \ 2, 7, 2,\ \ 2, 6, 2,\ \ 2, 5, 2]$ sent via `all_to_all_single` with no split. After the all-to-all, each rank learns how many value indices and length entries it will receive from every peer rank, and uses these sizes to preallocate receive buffers for Rounds 2 and 3.

Round 2 Length all-to-all. Rank 0 sends length subarrays $[2, 3]$, $[3, 4]$, $[4, 2]$, $[2, 3]$ to ranks 0–3 (`input_split_sizes=`$[2, 2, 2, 2]$), and receives $[2, 3]$ from Rank 0 itself for `EMT0`.

Round 3 Value all-to-all. Rank 0 dispatches index subarrays of sizes $[5, 7, 6, 5]$ to ranks 0–3, and receives value indices in four chunks (sizes determined by Round 1) from ranks 0–3, completing the redistribution of all lookup requests for `EMT0`.

A.3 Scale-Out With Rail-Optimized Topology

Specifically, when *GPU i* on node 0 sends messages (*e.g.*, sparse features) to *GPU j* on remote node 1, the communication backend first forwards them to the local *GPU j* on node 0 over the intra-node GPU interconnect. The local and remote *GPU j* share the same rail. Messages destined for the remote *GPU j* are aggregated at the local *GPU j*, which has the same local rank, and transmitted through the corresponding leaf switch, reducing small-message communication overhead. EMDedup also supports rail-optimized topologies by performing in-network deduplication at all leaf switches.

GPU i on each node connects exclusively to leaf switch L_i via its dedicated *NIC i*. Each leaf switch connects to every spine switch. When *GPU i* on node 0 sends data (*e.g.*, embedding requests) to *GPU j* on node 1, the communication backend first forwards the data to the local *GPU j* on node 0 over the intra-node GPU interconnect. The local and destination *GPU j* have the same local rank and share the same rail. All data destined for a given remote GPU are batched at the local GPU with the same local rank and transmitted through the corresponding leaf switch (Figure 7), reducing small-message communication overhead.

Input all-to-all. EMDedup applies rail-aligned forwarding and aggregation during the input all-to-all: sparse features destined for a given target rank are consolidated onto the GPU whose NIC shares a rail with that rank, then transmitted through that NIC. As a result, all traffic toward rail i passes exclusively through leaf switch i, and the feature sets

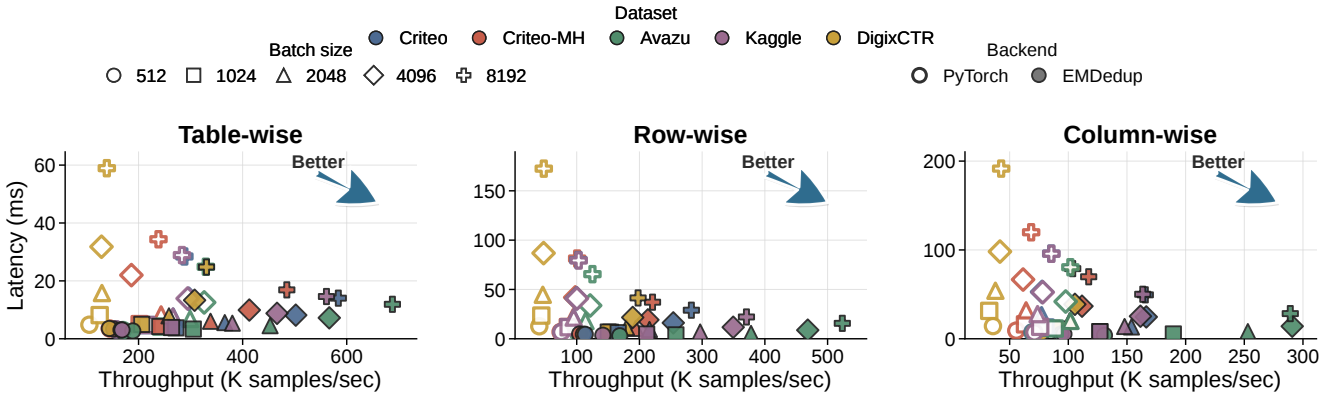


Figure 16. Latency–throughput tradeoff of EMDedup across three embedding parallelisms and five batch sizes.

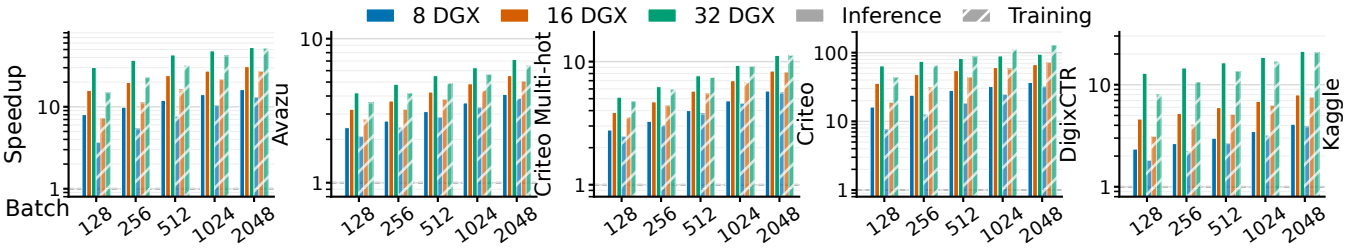


Figure 17. Speedup of EMDedup over PyTorch baseline under a ToR rack (one GPU per DGX node, one 100 Gbps NIC per GPU connected to a single switch), for cluster sizes of 8, 16, and 32 nodes and varying batch sizes, for both inference and training under row-wise EMT parallelism.

observed by distinct leaf switches are fully disjoint. This partitions the embedding layer logically into one independent shard per rail; each leaf switch runs EMDedup's deduplication and multicast logic autonomously on its own feature subset, equivalent to operating an independent small-scale ToR topology instance per rail.

Output all-to-all. Output all-to-all requires a different forwarding strategy. Under standard rail-aligned forwarding, RESP embeddings would be returned via each requesting GPU's home rail: in the example of Figure 7, node 0's GPU 0 (rail 0) and GPU 1 (rail 1) both sent sparse features to node 1's GPU 0 during the input phase, so a naive return path would route GPU 0's embeddings via leaf switch 0 and GPU 1's embeddings via leaf switch 1. However, leaf switch 1 never observed the corresponding REQs and holds no bitmap state; it cannot perform the required multicast, breaking correctness.

EMDedup instead applies a *request-path reversal* strategy: every embedding is transmitted back along the exact reverse of its request's inbound path, regardless of the recipient's home rail. In the example, node 1's GPU 0 routes all embeddings—for both node 0's GPU 0 and GPU 1—via rail 0, ensuring they pass through leaf switch 0, which retains the intact per-key bitmaps and multicasts them correctly. Node 0's GPU 0 receives all embeddings via its NIC and redistributes GPU 1's share locally over the intra-node GPU interconnect, shifting the aggregation role from sender to receiver.

Backward all-to-all. Gradient aggregation requires no special treatment. Node 0's GPU 0 and GPU 1 apply standard rail-aligned forwarding: both forward their gradients for a given key via rail 0 to node 1's GPU 0. Leaf switch 0, which retains the per-key bitmaps from the input phase, performs in-network gradient aggregation as described in §4.5. Together, these three phase-specific adaptations allow EMDedup to scale seamlessly to multi-switch rail-optimized topologies while preserving functional correctness.

A.4 Inference Throughput and Latency

EMDedup improves the throughput, *i.e.*, the number of requests served per second [43], as well as the latency over the baseline. As shown in Figure 16, table-wise parallelism achieves the best throughput-latency owing to its smaller overall communication volume, though at the cost of its limited scalability of. Between all datasets, Avazu benefits the most from EMDedup: its highly skewed feature access distribution yields a high request deduplication ratio, while

its compact model size maintains low per-request lookup and computation overheads. DigixCTR exhibits the highest latency and lowest throughput. This is attributed to its massive EMTs, thus incurring larger communication volume that dominates the inference runtime. Criteo Multi-hot ranks second in terms of overhead, where its large pooling size (up to 100 indices per feature) drives elevated lookup and communication costs.

A.5 Simulation results over ToR-Based Racks

We also evaluated a flat ToR rack topology in which each of 8, 16, or 32 nodes contributes one GPU connected via a 100 Gbps NIC to a single switch. This configuration is logically equivalent to a single domain partition of the rail-optimized topology. Figure 17 shows that the same trends hold: speedup increases with batch size, cluster size, and access skewness. Training speedup under ToR topology is modestly lower than under the rail-optimized topology. Because the flat configuration deploys only one GPU per node rather than eight, compute constitutes a larger share of total iteration time, shrinking the relative weight of the all-to-alls that EMDedup accelerates.